

TOPIC 2

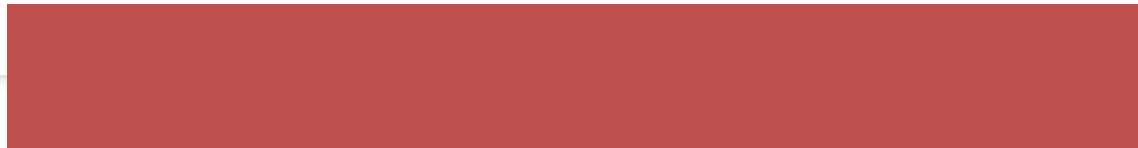
1. Activities in Analysis Phase
2. From Analysis to System Proposal
3. Information Gathering Techniques
4. Well-formed Requirements
5. Use Case

LEARNING OBJECTIVES

After this session, students will be able to:

- Explain the purpose and major activities of the analysis phase in the SDLC.
- Identify business problems, objectives, scope, stakeholders and user needs from a scenario.
- Select suitable information gathering techniques for a business case.
- Write clear functional and non-functional requirements.
- Derive actors, main use cases, system boundary, include/extend relationships and brief use case descriptions from functional requirements.

PART 1 - ACTIVITIES IN ANALYSIS PHASE



What is the Analysis Phase?

- Analysis phase is the stage in SDLC where analysts investigate the current situation, identify business needs, and define what the new system must accomplish.
- The focus is on **understanding the problem domain**, not on technical design.
- Main output: **documented requirements and analysis findings**

Role	Action
Analysis	What? Why? For whom?
Design	How? With what technology?

Why Analysis Before Design

- Poor analysis leads to wrong system design
- Early design without valid requirements increases rework cost
- Analysis reduces ambiguity, uncovers hidden issues, and aligns the project with business goals
- Example: Building attendance system without understanding rules → incorrect logic

Analysis Phase Flow

The analysis phase follows a logical sequence of activities that move from understanding the problem to preparing a justified recommendation.

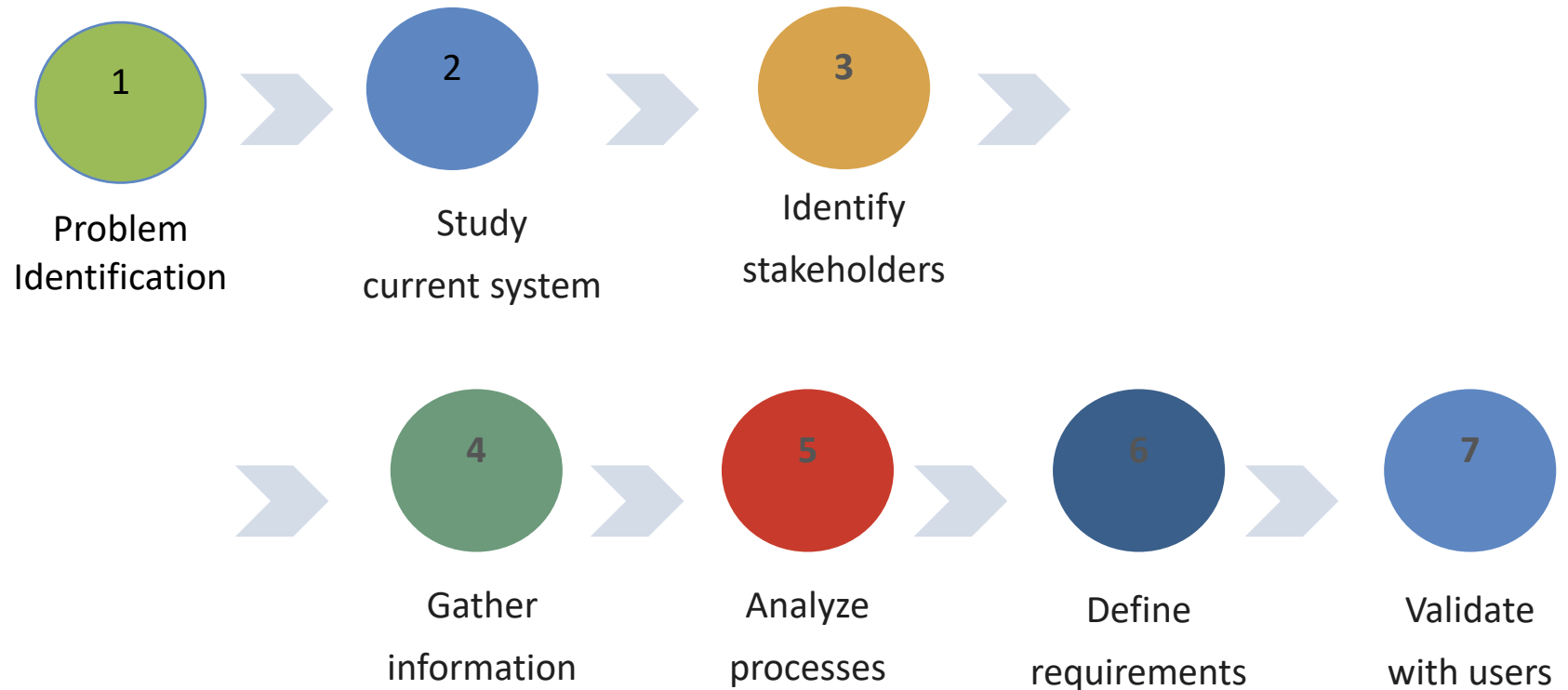
Typical analysis flow

- Identify the business problem or opportunity
- Study the current system and work practices
- Gather information from users, documents, and observations
- Analyze processes, issues, and business rules
- Define and organize system requirements
- Validate findings with stakeholders
- Prepare a system proposal for decision making

Key idea:

Analysis is not a random collection of tasks. It is a structured process that transforms business understanding into documented requirements and proposal inputs

Analysis Phase Flow



Main Activities in the Analysis Phase

A. Understanding the current business situation

1. Study current system
2. Identify business problems and opportunities
3. Identify stakeholders

B. Investigating and documenting

1. Gather information
2. Analyze workflows, documents, and data usage
3. Analyze business processes

C. Defining and validating the solution direction

1. Define requirements
2. Validate findings with users
3. Prepare analysis deliverables and proposal inputs

Studying the Current System

- Analysts examine how work is currently performed
- They look at documents, approvals, data movement, delays, manual effort, and recurring errors
- The current system may be manual, partially digital, or fully computerized.
- Understanding the current system helps avoid repeating existing weaknesses in the new system

Example:

In a university office, students may need to fill paper forms, wait in line, and return later because request handling is manual and inconsistent

How to identify a problem

- A problem is not just something inconvenient.
- A strong problem should show:
 - Current condition
 - Evidence from scenario
 - Impact on users or organization
 - Possible cause or related process issue

Example:

Evidence from scenario	Problem	Impact
Staff record requests in separate Excel files	Fragmented request records	Requests may be lost or duplicated
Students contact office to ask status	Lack of status visibility	More workload for staff and poor student experience

Finding Problems in the Current Process

After documenting the current process, analysts identify specific weaknesses that affect business performance and user experience.

Common process problems include

- Repeated delays in completing tasks
- Duplicated data entry or repeated paperwork
- Missing or incomplete information
- Unclear approval steps or responsibilities
- Communication gaps between departments
- Difficulty tracking status or progress

These findings are important because they become evidence for requirement definition and system recommendation.

Finding Problems in the Current Process

Example:

If students cannot track the status of a request and staff use separate files to manage records, the analyst may identify poor transparency and fragmented data handling as major process problems.

Key idea:

The stronger and more concrete the problem evidence is, the stronger the later system proposal becomes.

Identifying Problems and Opportunities

- **Problem** = undesirable current condition
- **Opportunity** = possible improvement area

Example:

- Problem: long processing time, repeated data entry, missing request tracking
- Opportunity: self-service submission, online tracking, automatic notification

Identifying Stakeholders and Users

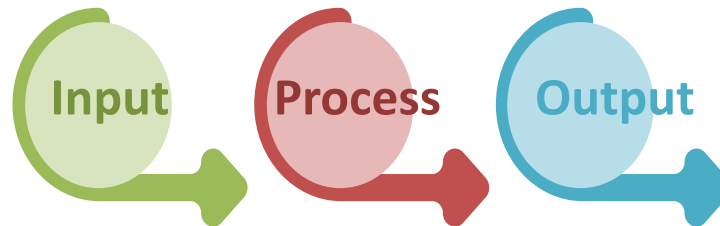
- Analysts must identify who uses the system, who manages it, who receives output, and who approves major changes.
- Different users may need different features, screens, and reports.
- Good documentation records these user groups clearly.
- **Typical stakeholder groups**
 - primary users
 - operational staff
 - managers / approvers
 - technical support / administrators
 - external or related departments

Example: in payroll, HR staff, accountants, employees, and managers all interact differently with the system.

Documenting Business Processes

- A business process is a set of related activities that transform inputs into outputs to achieve an organizational objective.
- During analysis, analysts document how work flows from one step to another, who performs each step, and what information is used.
- This helps reveal inefficiency, duplication, delay, and unnecessary manual effort.

Simple process view:



Example:

Request form → verification and approval → issued academic document

Analysis Deliverables

- A deliverable is a useful output produced during the analysis phase. These deliverables help the team move from raw information to shared understanding and formal decision making.

Common analysis deliverables include

- Interview notes and questionnaire results
- Observation records and document analysis summaries
- Stakeholder list and problem summary
- Requirement list and business rules
- Process models, diagrams, or use case descriptions
- Analysis report and system proposal

Analysis Deliverables

Why deliverables matter

- They provide evidence for recommendations
- They help stakeholders review and validate findings
- They support the transition from analysis to design

Key idea:

Good analysis deliverables are clear, organized, traceable, and useful for both communication and decision making.

Activities in the Analysis Phase

The analysis phase is an investigative and evidence-based stage of system development.

Activity	Purpose	Typical Output
Study current system	understand present operations	process notes
Identify business problems	find pain points	problem list
Identify stakeholders	know who matters	stakeholder list
Gather information	collect evidence	interview/survey data
Define requirements	specify system needs	requirement statements
Validate findings	confirm accuracy	reviewed analysis notes
Produce deliverable	support the system proposal	the system proposal

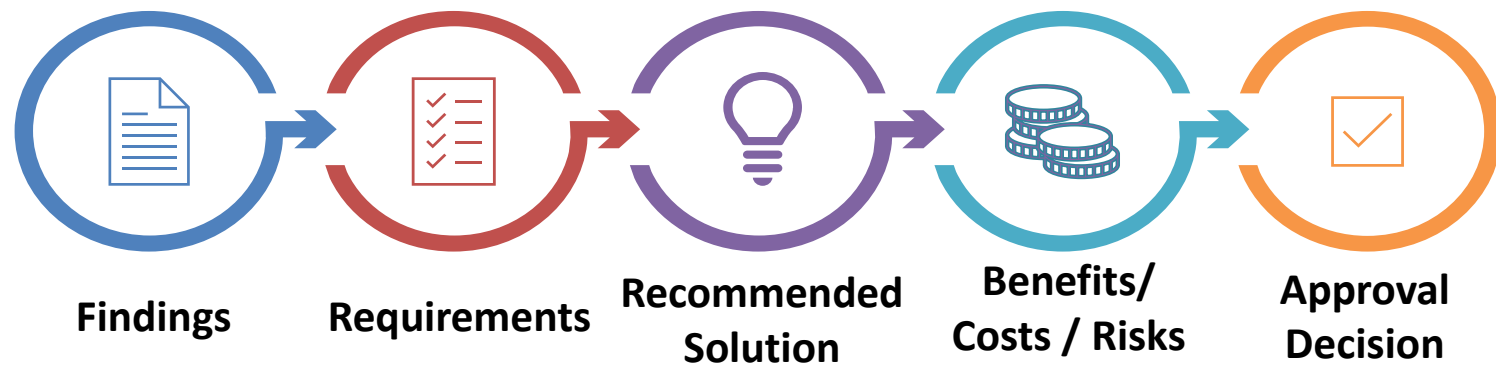
The main goal of analysis is to understand what the system must do, why it is needed, and how the current situation should be improved before design begins.

PART 2 - FROM ANALYSIS TO SYSTEM PROPOSAL



From Analysis to System Proposal

- After collecting and organizing information, the analyst prepares a system proposal.
- A proposal should not be based on personal opinion; it should be based on evidence and analysis findings.
- The proposal connects business problems, requirements, and recommended actions.



What Is a System Proposal?

- A system proposal is a formal document or presentation that summarizes analysis results and recommends a solution.
- It is used for decision making and approval before moving to the next stages of the project.
- A strong proposal should be clear, realistic, evidence-based, and understandable to both technical and non-technical stakeholders.

Main roles of a system proposal

- Communication tool
- Decision support document
- Transition document from analysis to design

Why the Proposal Matters

A system proposal is important because it:

- Helps management decide whether the project should continue
- Explains the value of the proposed system
- Clarifies scope, objectives, and expectations
- Communicates major benefits, costs, and risks
- Reduces misunderstanding before design and implementation begin

The proposal turns analysis into management action.

Common Sections of a System Proposal

A system proposal usually contains several sections that explain the current problem, the recommended direction, and the justification for moving forward.

Common sections include

1. Problem statement
2. Project objectives
3. Project scope
4. Requirement summary
5. Recommended solution
6. Expected benefits
7. Estimated costs
8. Major risks and assumptions
9. Implementation considerations or next steps
10. Approval request

Although the exact structure may vary by organization, the purpose remains the same: to justify and recommend a practical solution based on analysis findings.

Common Sections of a System Proposal

A system proposal usually contains several sections that explain the current problem, the recommended direction, and the justification for moving forward.

Common sections include

1. Problem statement	6. Expected benefits
2. Project objectives	7. Estimated costs
3. Project scope	8. Major risks and assumptions
4. Requirement summary	9. Implementation considerations or next steps
5. Recommended solution	10. Approval request

Although the exact structure may vary by organization, the purpose remains the same: to justify and recommend a practical solution based on analysis findings

Structure of System Proposal

A strong system proposal should be structured in a logical way so that readers can understand the problem, the proposed direction, and the expected business value.

Suggested structure

- **Problem Statement:** clearly describe the current issue and its impact
- **Objectives:** state what the new system should achieve
- **Scope:** define what the project includes and excludes
- **Requirement Summary:** summarize key user and business needs
- **Recommended Solution:** present the proposed system direction
- **Benefits:** explain financial and non-financial value
- **Costs:** estimate development, training, and maintenance costs
- **Risks and Assumptions:** identify major uncertainties or challenges
- **Next Step:** request approval or indicate the next project action

Key idea:

A proposal is stronger when each section is supported by evidence from the analysis phase.

From Problem to Proposal

A system proposal connects analysis findings to a decision.

A good proposal should answer:

- What problem exists in the current situation?
- Why does the problem matter?
- What objectives should the new system achieve?
- What is included and excluded in the project scope?
- What key requirements support the proposed solution?
- What benefits, costs and risks should management consider?

Key idea:

A proposal is not a personal opinion. It should be supported by analysis evidence.

Problem Statement in Proposal

- A problem statement explains what is wrong in the current situation and why it matters to the organization
- A good problem statement should be specific, factual, and linked to business impact.
- Vague statements are weak because they do not justify change clearly

Strong example	Weak example
Manual handling of academic document requests causes long waiting time, inconsistent processing, and limited status visibility for students and staff.	The current system is inconvenient.

Problem Statement in Proposal

A problem statement explains what is wrong in the current situation and why it matters to the organization

A good problem statement should be

- Be specific and factual
- Describe the current issue clearly
- Show business or service impact
- Avoid vague language such as “bad”, “slow”, or “inconvenient” without evidence

Strong example	Weak example
Manual handling of academic document requests causes long waiting time, inconsistent processing, and limited status visibility for students and staff.	The current system is inconvenient.

Key idea:

A clear problem statement gives a strong reason for change and helps justify the proposed system

Objectives in a Proposal

Objectives describe the expected improvement, not the detailed system function. They should be clear and measurable whenever possible.

Good objectives often focus on business results:

- Reduce processing time.
- Improve request tracking.
- Increase data accuracy.
- Support better reporting.
- Improve service transparency

Examples

- Problem: Students cannot track request status.
- Objective: Improve transparency of request processing.

Scope in a Proposal

Scope defines what the project will include and what it will exclude.

A clear scope is important because it:

- Prevents confusion and unrealistic expectations
- Helps control project size and complexity
- Keeps the project aligned with its objectives

In scope	Out of scope
Online request submission	Full student information system replacement
Request status tracking	Hostel management
Automated notification	Unrelated financial management function
Digital approval flow for selected request types	

A project becomes easier to manage when the scope is clearly defined and approved early.

Benefits, Costs, and Risks in a Proposal

A realistic proposal should present both the value of the new system and the challenges involved in implementing it.

Benefits	Costs	Risks
Faster service and shorter processing time	Software development or acquisition	Resistance to change from users or staff
Fewer manual errors and duplicated work	Staff training and change support	Incomplete or misunderstood requirements
Better transparency and request tracking	Maintenance and technical support	Implementation delays
Improved user satisfaction and service quality	Staff time and effort required for transition	Integration difficulties with existing systems

A good proposal is balanced. It should not only promise benefits but also explain realistic costs and risks.

MINI CASE: TURNING FINDINGS INTO PROPOSAL ELEMENTS

The table below shows how analysis findings can be transformed into requirements and proposal content.

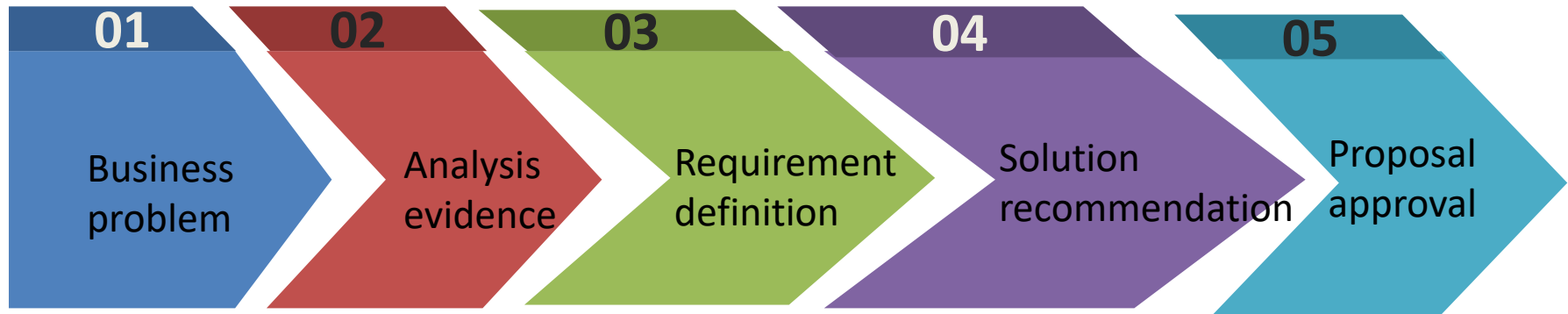
Analysis Finding	Problem	Objective	Requirement
Students must visit the office physically	Manual submission causes inconvenience	Support online service access	The system shall allow students to submit service requests online
Request status is unclear	Students cannot track progress	Improve request transparency	The system shall allow students to view request status

MINI CASE: TURNING FINDINGS INTO PROPOSAL ELEMENTS

Analysis Finding	Problem	Objective	Requirement
Records are stored in separate files	Request data is fragmented	Centralize request records	The system shall store all service requests in a central database.
Approval depends on manual contact	Processing is inconsistent	Support controlled approval workflow	The system shall allow staff to approve, reject, or request additional information.

Analysis findings become requirements, and requirements become proposal elements.

Summary: From Analysis to Proposal



- A system proposal is not a guess. The system proposal is the formal result of analysis work.
- It is a structured recommendation built on evidence collected during analysis.

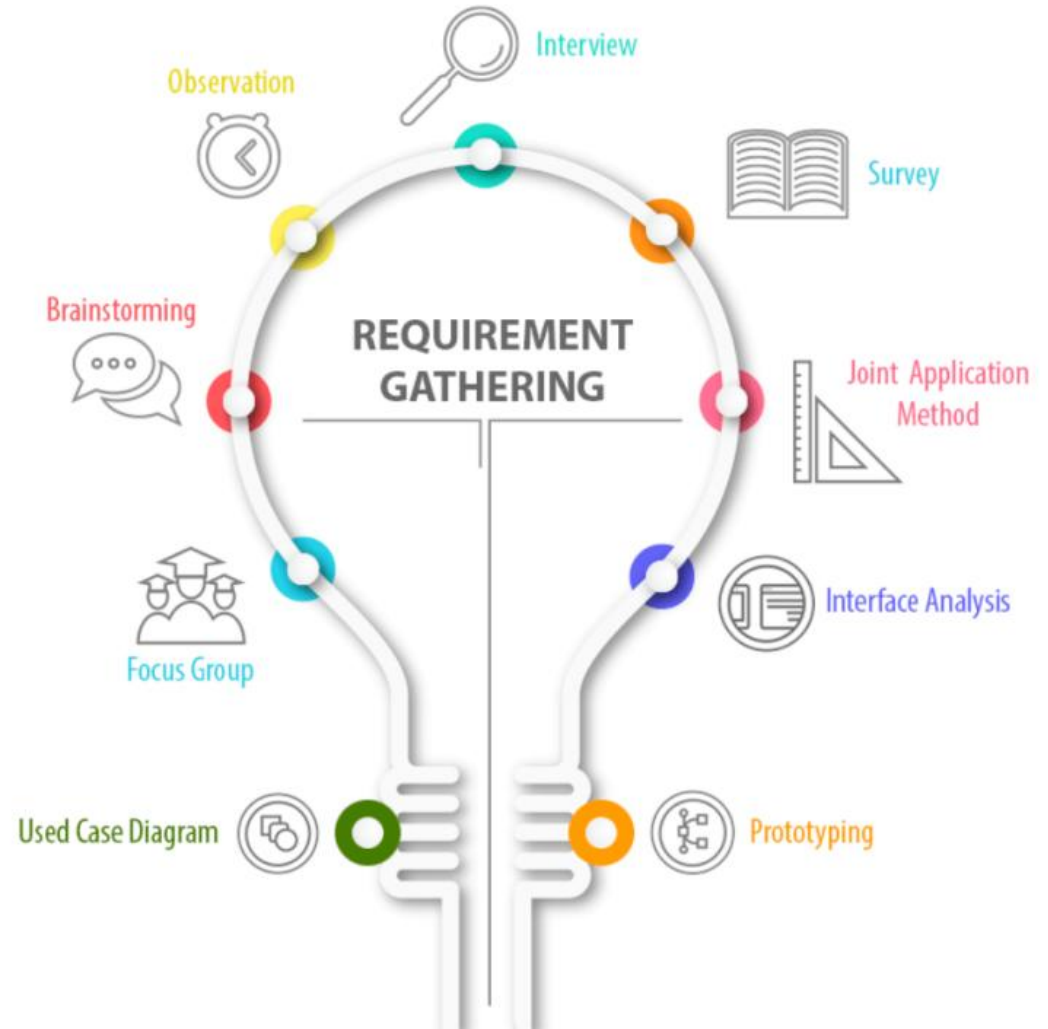
Summary: From Analysis to Proposal

Logic of this part:

- Business problems are identified during analysis
- Evidence is collected through investigation and information gathering
- Findings are translated into requirements
- Requirements support a recommended solution
- The proposal presents benefits, costs, and risks for decision making

A system proposal is not based on guesswork. It is a structured recommendation built on evidence collected during the analysis phase

PART 3 - INFORMATION GATHERING TECHNIQUES



Information Gathering: Meaning

- Information gathering is the process of collecting facts, opinions, observations, and documents needed to understand the current system and define requirements.
- It is a core responsibility of the systems analyst during the analysis phase.
- Good analysis depends on the quality, accuracy, and completeness of gathered information.

Key terms: fact finding, elicitation, evidence, validation

Why Multiple Techniques Are Needed

- One technique alone rarely gives the full picture.
- Different methods reveal different kinds of information.
- Combining methods improves completeness, reliability, and accuracy.

Examples

- Interviews show opinions and explanations
- Observation shows actual behavior
- Documents show formal rules and data
- Questionnaires show patterns from large groups

Main Information Gathering Techniques

Common techniques include:

- Interviews
- Questionnaires / Surveys
- Observation
- Document Analysis
- Sampling
- Workshops / JAD Sessions
- Analysts choose techniques based on purpose, time, number of users, and complexity of the problem.

Interviews

- An interview is a planned conversation used to gather detailed information from users or stakeholders.
- It is useful when the analyst needs explanation, clarification, opinions, goals, or hidden issues.
- Good interviews require preparation, clear questions, active listening, and follow-up questions.

Strengths

- Rich detail
- Flexible
- Can uncover hidden problems

Limitations

- Time-consuming
- Depends on interviewer skill
- Responses may be subjective

Open and Closed Interview Questions

Analysts use both open and closed questions during interviews because each type serves a different purpose.

Open questions	Closed questions
Invite explanation and discussion	Ask for short, fixed answers or specific answers
Help explore opinions, experiences, and problems	Help confirm facts, quantities, or simple conditions
Usually produce longer answers	Useful for confirming facts
Example: What difficulties do you face when processing student requests?	Example: How many request forms do you process per day?

Analysts often begin with open questions to explore issues and then use closed questions to confirm details.

Questionnaires / Surveys

A questionnaire is a written set of questions distributed to many users to collect information efficiently.

It is useful when:

- The analyst needs broad feedback from a large group
- Users are in different locations
- Time is limited
- The goal is to identify trends or patterns

Questionnaires / Surveys

Strengths	Limitations
Reaches many users	Limited depth of explanation
Fast and cost-effective	Difficult to clarify misunderstood questions
Useful for quantitative data and comparison	Response rate may be low

Example:

Surveying 500 students about difficulties in course registration or service requests.

Observation

- Observation means watching users perform real work in the real environment.
- It helps analysts see actual behavior, delays, interruptions, workarounds, and hidden steps.
- Observation is especially useful for operational or service processes.

Important idea:

Users may describe what should happen, but observation shows what actually happens.

Strengths	Limitations
Reveals real practice	Time-consuming
Identifies hidden steps	Users may change behavior when watched
Captures context	Not all decisions are visible

Document Analysis

- Document analysis means reviewing existing forms, reports, manuals, policies, logs, spreadsheets, and database outputs.
- Documents help analysts identify required data, business rules, approvals, and outputs.
- They provide useful formal evidence about how the organization officially works.

Example:

Analyzing request forms to determine mandatory fields, approval steps, and required attachments.

Strengths	Limitations
Based on existing evidence	Documents may be outdated
Useful for identifying data and rules	May not reflect real practice fully
Less disruptive than interviews	

Sampling Existing Records

- Sampling means reviewing a small but representative set of records instead of checking everything.
- It saves time while still revealing common patterns, frequent errors, and repeated issues.
- Sampling is useful when the organization has too many records to inspect completely.

Example:

Reviewing 100 student service requests from the previous semester to identify common request types and delays.

Workshops and JAD Sessions

- A workshop or JAD session is a structured group meeting where users, managers, and analysts discuss requirements together.
- It is useful when many stakeholders must align quickly on issues, priorities, or requirements.
- These sessions need preparation, facilitation, and clear goals.

Strengths

- Faster agreement
- Shared understanding
- Efficient for cross-functional discussion

Limitations

- Requires coordination
- Can be difficult if participants disagree strongly
- Depends on facilitation quality

Comparison of Information Gathering Techniques

Different techniques are suitable for different purposes. No single method is best for every situation.

Technique	Best for	Strength	Limitation
Interview	depth and clarification	detailed insight	time-consuming
Questionnaire	broad feedback	reaches many users quickly	limited depth
Observation	real work behavior	shows actual practice	may require much time
Document Analysis	formal rules and data	provides existing evidence	may not reflect real practice fully
Sampling	pattern identification	efficient review of many records	only part of total data is checked
Workshop / JAD	stakeholder alignment	shared understanding	requires coordination

No single method is best for every situation.

Choosing the Right Technique

- Analysts select techniques based on the kind of information needed.
- Need detailed explanation? → **Interview**
- Need responses from many users? → **Questionnaire**
- Need to see actual work? → **Observation**
- Need existing rules or data fields? → **Document analysis**
- Need common patterns from many records? → **Sampling**
- Need agreement among stakeholders? → **Workshop / JAD**

Applying Techniques to a Case

Case: University Student Service Request System

The analyst should match each information need with the most suitable source and technique.

Information Needed	Best Source	Best Technique
student frustrations and expectations	students	questionnaire / interview
actual processing delays	office workflow	observation
approval rules and required fields	forms, policies, and records	document analysis
common request types and repeated issues	past request records	sampling
agreement on future improvements	staff and managers	workshop / JAD

The quality of analysis improves when the analyst selects techniques based on the type of information needed

Common Mistakes in Information Gathering

- Common mistakes include:
- asking vague or leading questions
- interviewing only one stakeholder group
- accepting opinions without validation
- ignoring actual work practices
- collecting too much data without clear structure
- failing to document findings clearly

Lesson:

Good information gathering requires planning, balance, and validation.

Summary: Information Gathering Techniques

Information gathering → evidence → analysis → requirements

- Information gathering is the foundation of requirement discovery.
- During analysis:
 - Analysts collect facts, opinions, observations, and documents
 - Different methods provide different perspectives
 - No single technique is enough for every situation
 - Combining techniques improves completeness and reliability
- Different methods provide different perspectives.
- Strong analysis depends on choosing the right techniques, validating findings, and turning gathered information into useful requirements.

Guided Activity:

From Problem to Requirement

Case scenario

A university office handles transcript requests manually. Students must visit in person, staff record requests in separate files, and processing time is inconsistent.

Use the case to **identify**:

- Current business problems.
- Evidence from the scenario.
- Expected objectives.
- Possible user needs.
- Candidate functional requirements.

Final idea:

A good requirement should come from a real problem, user need, business rule, or improvement objective.

Activity: Small Case Discussion

Discuss the following questions

- What are the main business problems in this case?
- Who are the key stakeholders?
- What information is still needed before writing requirements?
- Which information gathering techniques should be used first, and why?
- What system objectives may be proposed?

Activity goal:

Apply analysis thinking to a realistic business case.

Case Study

University Student Service Request System

To analyze this system effectively, the analyst should ask:

- What services are students requesting most often?
- What information is required for each request?
- How are requests currently submitted, checked, and approved?
- Where do delays or errors usually occur?
- How do students know the status of their requests?

Possible interviewees

- Students
- Academic affairs staff
- Department approvers
- System administrator

Key idea:

A good case analysis starts by identifying what information is needed and who can provide it.

Activity (Role-play)

Roles

- Group A = systems analyst
- Group B = client or user representative

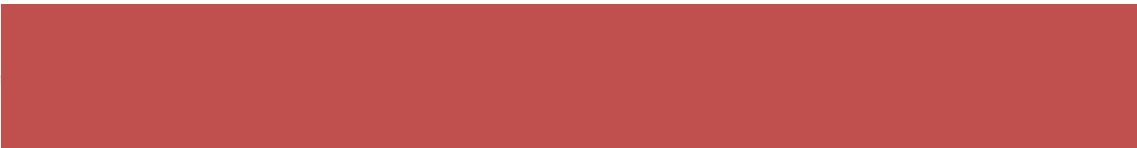
Task

Conduct a short interview about the current request handling process.

The analyst must

- Ask at least 5 relevant questions
- Use both open and closed questions
- Identify current problems
- Identify user needs or expectations
- Record key findings clearly

PART 4 - WELL-FORMED REQUIREMENTS



Requirements turn analysis evidence into clear statements of what the system must do and what conditions it must satisfy.

WHAT IS A REQUIREMENT?

- **A requirement is a statement describing what the system must do or what condition the system must satisfy.**
- Requirements guide design, coding, testing, and acceptance later.

Types of requirements

****Functional requirement****

****Non-functional requirement****

A good requirement should be written in a way that different readers interpret it in the same way.

FUNCTIONAL VS NON-FUNCTIONAL REQUIREMENTS

Functional Requirements

- Describe system services, actions, or behavior.
- Often derived from user goals and business processes.
- Example: The system shall allow students to submit service requests online.

Key idea:

Functional = what the system does.

Non-functional = how well or under what condition the system performs.

Non-functional Requirements

- Describe quality expectations or constraints.
- Common areas: performance, security, reliability, usability, availability.
- Example: The system shall respond to search requests within two seconds.

Why requirements matter

In systems analysis, requirements are the foundation for every later activity.

If requirements are unclear, incomplete, or inconsistent, the system may be designed incorrectly, even if developers work very hard.

A good requirement helps analysts, users, designers, and developers share the same understanding of what the system must do.

A system fails not only because of bad coding, but often because of bad understanding.

Requirements are important because they:

- Define what problem the system must solve
- Identify what users need from the system
- Guide later design and testing
- Reduce misunderstandings between stakeholders

What is a well-formed requirement?

A **well-formed requirement** is a requirement that is written clearly enough that different readers interpret it in the same way.

Common characteristics of a well-formed requirement:

- 1. Clear**
Easy to understand, avoids vague words.
- 2. Specific**
It states exactly what the system should do.
- 3. Testable / Verifiable**
Someone can check whether the requirement has been satisfied.
- 4. Complete**
It contains enough information to be useful.

What is a well-formed requirement?

5. Consistent

It does not conflict with other requirements.

6. Feasible

It can realistically be implemented with available time, technology, and budget.

Example:

- The system shall allow students to submit a transcript request online.
- The system shall send a notification email after a request is submitted.
- The system shall store request status for staff tracking.

Writing Requirement Statements

Use a simple structure:

The system shall + action + object + condition/constraint.

- Good: The system shall notify the student when a service request is approved, rejected, or completed.
- Weak: The system should be convenient and fast.
- Avoid vague words unless they are measurable: fast, user-friendly, easy, soon, many, secure.
- Link each requirement to a stakeholder need, business rule, or analysis finding.

COMMON REQUIREMENT MISTAKES

- Mixing requirement with design: “Use a blue button” before interface design is discussed.
- Writing too much technical implementation detail during analysis.
- Writing design decisions as requirements too early.
- Using vague words such as fast, easy, convenient, many, or secure without measurement.
- Missing business rules, validation conditions, or exception cases.
- Writing requirements that cannot be tested.
- Not linking requirements to problems, stakeholders, or evidence.
- Not checking requirements with users before moving to diagrams.

From problem statement to requirements

Step 1: Identify the problem

- What is wrong in the current process?

Step 2: Identify the goal

- What should the new system improve?

Step 3: Identify users

- Who will use the system or receive outputs from it?

Step 4: Extract required functions

- What must the system do to support those users?

Main Functional Requirements

A main functional requirement should:

- Describe a major service the system must provide;
- Support a user goal or business process;
- Be traceable to a problem, objective, stakeholder need, or business rule;
- Be possible to convert into one or more use cases.

Example:

- The system shall allow students to submit service requests online.
- The system shall allow staff to review submitted requests.
- The system shall allow staff to approve, reject, or request additional information.
- The system shall notify students when request status changes.
- The system shall allow managers to view request summary reports.

Main Functional Requirements

A main functional requirement describes a major service that supports a user goal or core business process.

Main functional requirements are usually related to:

- Submitting, creating, or recording important business data;
- Reviewing or processing a request;
- Approving, rejecting, updating, or tracking status;
- Searching or viewing important information;
- Generating reports or notifications.

Supporting functions such as login, validation, and notification may be modeled as included or supporting behavior depending on the diagram purpose.

Main case study

Case Study: Student Academic Service Request System

At a university, students currently go to the Academic Affairs Office in person to request documents such as transcripts, enrollment certificates, student letters, replacement student ID cards, and graduation confirmation documents. Staff record requests manually in notebooks or separate Excel files. During busy periods, requests are delayed, some documents are missing, and students cannot easily track progress.

Goal

- Develop a digital system that allows students to submit service requests online and allows staff to process and track them consistently.

Main case study

Current problems

- Students must visit the office in person
- Staff record requests manually
- Requests are handled inconsistently
- Students cannot track progress easily

Target users

- Students
- Academic Affairs Staff

Expected improvements

- Online request submission
- Better request tracking
- Faster processing
- Consistent handling and reporting

Identify requirement statements from main case study

Functional requirements:

- The system shall allow students to select a service type.
- The system shall allow students to submit a service request online.
- The system shall allow students to upload supporting documents when required.
- The system shall record request date, request type, requester information and request status.
- The system shall allow students to view current request status after login.
- The system shall allow staff to review submitted requests.
- The system shall allow staff to approve, reject, or request additional documents.
- The system shall allow staff to search requests by student ID or request code.
- The system shall allow managers to view request summary reports.

Non-functional / quality requirements:

- The system shall send a confirmation message within 1 minute after successful submission.
- The system shall restrict request records to authorized users only.
- The system shall display request status within 2 seconds under normal operating conditions.

Problem vs Objective vs Requirement

Type	Meaning	Example
Problem	What is wrong now?	Students cannot track request status.
Objective	What improvement is expected?	Improve transparency of request processing.
Functional Requirement	What must the system do?	The system shall allow students to view request status.
Use Case	User goal supported by system	View Request Status

Key idea:

A use case should normally be traceable to at least one functional requirement.

PART 5 – USE CASE

Use cases describe how actors interact with the system to achieve user goals. They make functional requirements easier to discuss and validate.

WHAT IS A USE CASE?

- A use case describes a goal-oriented interaction between an actor and the system.
- It focuses on what the user wants to achieve, not how the interface or code will be implemented.
- Use cases help analysts organize functional requirements around user goals.
- A use case is:

a user goal + the system response

Examples: Submit Request, Approve Request, Search Course, Generate Report.

What is an actor?

- An actor is a person, organization, device, or external system that interacts with the system to achieve something of value.
- An actor is outside the system boundary.
- An actor represents a role, not one specific person

An actor is a role, not simply a job title.

Examples:

- Student
- Academic Affairs Staff
- Manager
- Email/SMS Notification Service, if it is an external service connected to the system.

A person can play multiple roles in different situations

Requirement to Use Case Mapping

Functional Requirement	Actor	Candidate Use Case
The system shall allow students to submit service requests online.	Student	Submit Service Request
The system shall allow staff to review submitted requests.	Staff	Review Request
The system shall allow staff to approve or reject requests.	Staff	Approve/Reject Request
The system shall notify students when status changes.	External Notification Service/System	Notify Student
The system shall allow managers to view request summary reports.	Manager	View Request Report

Identify use cases from the case study

Student	Staff	Supporting
Submit Service Request	Review Submitted Requests	Login
Upload Supporting Documents	Verify Documents	Notify Student
View Request Status	Request Additional Documents	
View Request History	Approve/Reject Request	
Download Result / Collection Notice	Update Request Status	

System boundary

The system boundary defines what is inside the system and what is outside.

In a use case diagram, **actors** stay **outside** the system boundary, while **use cases** stay **inside** the system boundary.

The boundary answers this question:

“Is this part of the system we are building, or is it outside the system?”

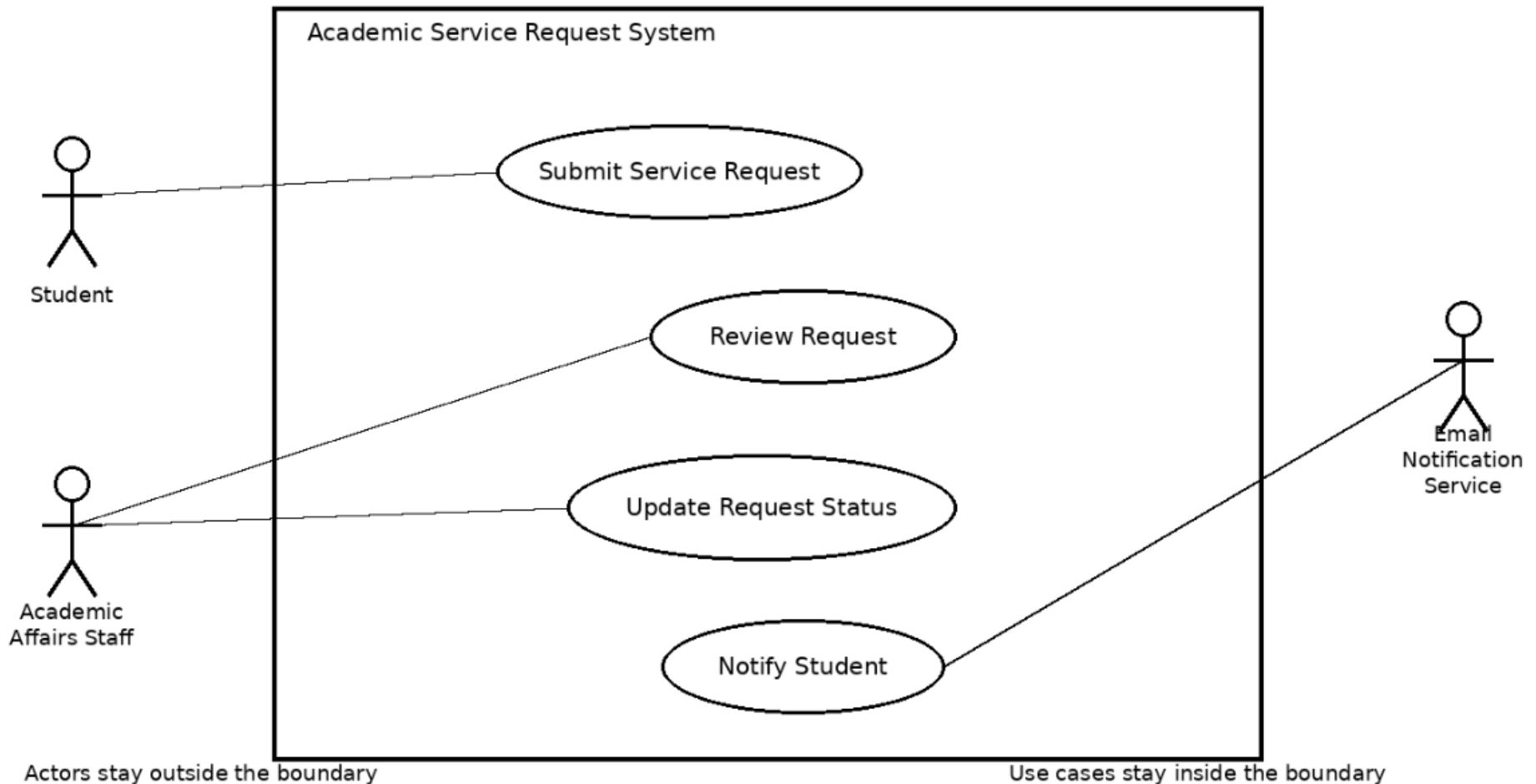
Example: **Inside the boundary:**

- Submit Service Request
- Upload Supporting Documents
- Review Submitted Request
- Verify Documents
- Approve/Reject Request
- Update Request Status
- View Request Status
- Notify Student

Outside the boundary:

- Student
- Academic Affairs Staff
- Notification Service/System

System boundary



How to name use cases correctly

A good use case name should:

- Start with a verb
- Describe a user goal
- Be short and meaningful

Use case should represent **something the actor wants to achieve**.

Good examples	Weak examples
Submit Request	Request
Verify Documents	Student Function
Approve Request	Document Process
View Request History	Information

Relationships in use case diagrams

1. Association

A basic link between an actor and a use case.

Example:

- Student — Submit Request
- Staff — Review Request

2. Include

Use **include** when one use case always uses another required behavior.

Example:

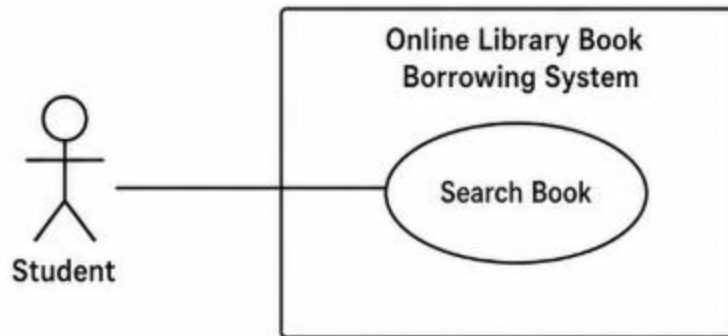
- Submit Service Request <<include>> Validate Required Information
 - Approve/Reject Request <<include>> Update Request Status t
- >>> Include = **mandatory reused behavior**

Avoid:

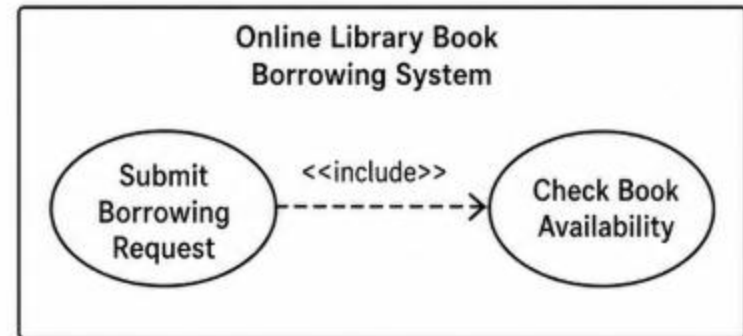
Do not use <<include>> for optional behavior.

Association and Include

1. Association



2. <<include>>



Relationships in use case diagrams

3. Extend

Use <<extend>> when an optional or conditional behavior may happen.

Example:

- Upload Supporting Documents <<extend>> Submit Service Request
 - Request Additional Documents <<extend>> Review Submitted Request
- >>>> Extend = **optional or conditional behavior**

4. Generalization

Use when one actor or use case is a specialized form of another.

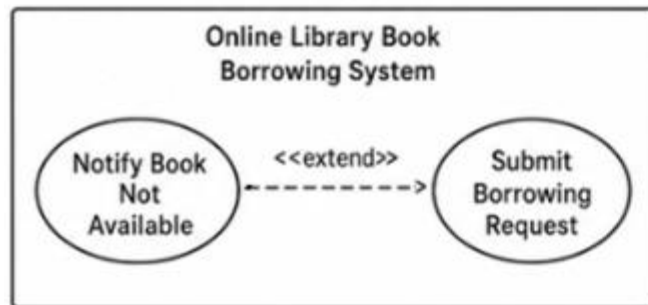
Example:

- Staff may be generalized into Senior Staff and Front Desk Staff
- Make Payment may have specialized forms such as Pay by Card and Pay by E-wallet

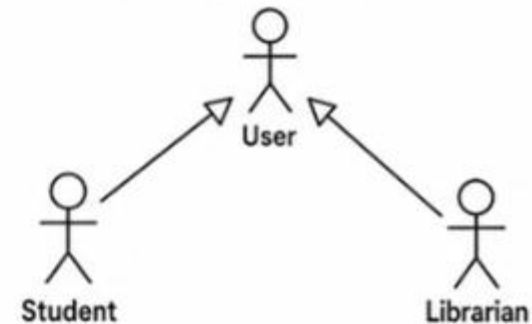
Generalization = **inheritance / specialization**

Extend vs Generalization

3. <<extend>>



4. Generalization



Use Case Description Template

Field	Example
Use case name	Submit Service Request
Primary actor	Student
Goal	Submit an academic service request online
Trigger	Student wants to request an academic document
Preconditions	Student is logged in
Main flow	Select service type → enter information → attach documents if required → submit request → system records request
Alternative flow	Missing required document → system asks student to upload document
Postcondition	Request is saved with Submitted status

Mini example to explain include vs extend

Base use case: Submit Service Request

Include

- Validate Required Information
- This must always happen, so it is included.

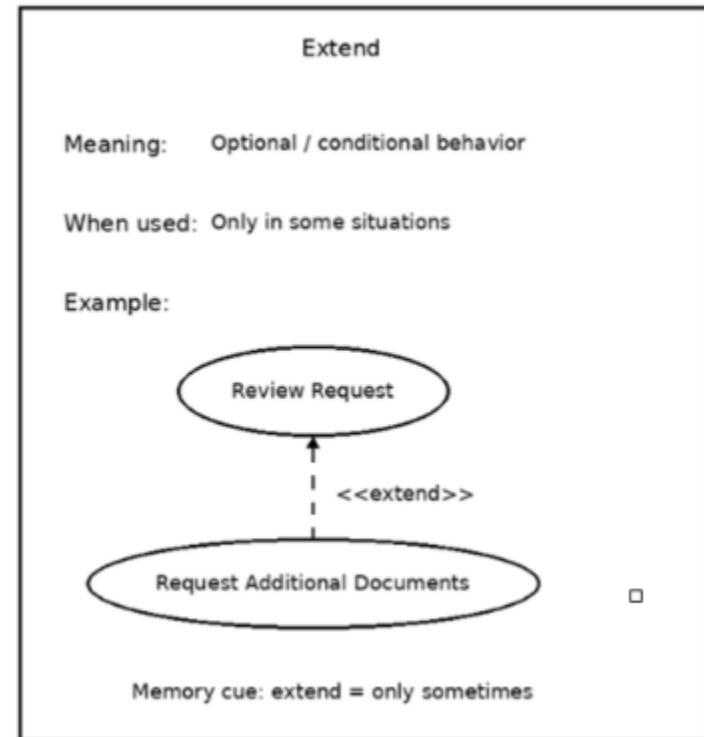
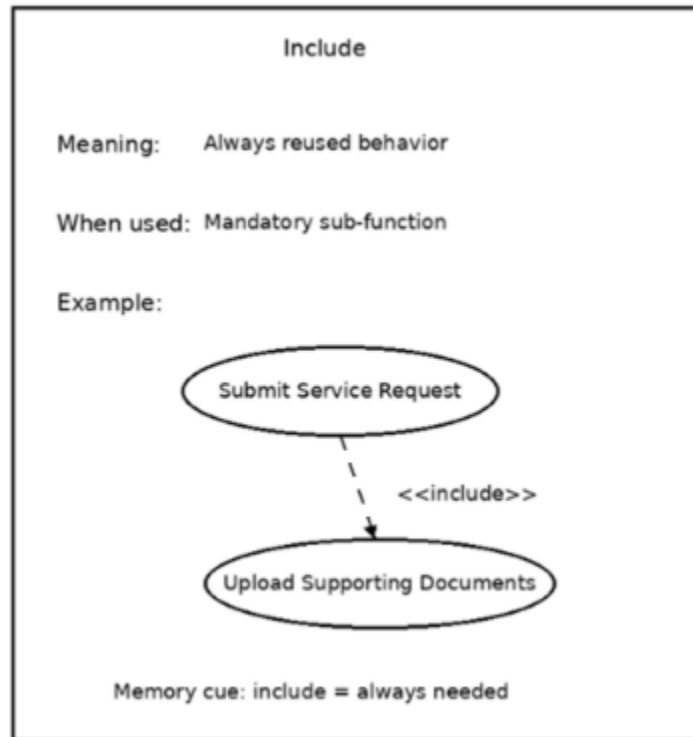
Extend

- Upload Supporting Documents
- This only happens when the student chooses urgent processing, so it extends the base use case

include = always needed

extend = only sometimes

Include vs extend Comparison



Common mistakes when deriving use cases

- Naming use cases with nouns instead of verb phrases.
- Placing actors inside the system boundary.
- Drawing database, screen, button, or internal module as an actor.
- Using <<include>> for optional behavior.
- Using <<extend>> for mandatory behavior.
- Missing associations between actors and use cases.
- Creating use cases that cannot be traced back to functional requirements.
- Treating every small step as a separate use case.

Mini case

Scenario

Students can book study rooms online. They can search available rooms, make a reservation, cancel a reservation, and receive a confirmation email. Library staff can approve special booking requests and update room availability.

Task

1. Identify actors.
2. Identify 4–6 main functional requirements.
3. Map each functional requirement to a candidate use case.
4. Draw a simple use case diagram.
5. Write a brief use case description for one main use case.

Conclusion

In this topic, we learned that:

- The analysis phase focuses on understanding business problems, users, evidence and system needs before design begins.
- Information gathering helps analysts collect facts, opinions, documents and observations.
- Problems should be supported by evidence and linked to objectives.
- Well-formed requirements describe what the system must do or what quality it must satisfy.
- Main functional requirements can be transformed into actors, use cases and brief use case descriptions.
- A good use case diagram should show actors, system boundary, main use cases, associations and suitable include/extend relationships.

Keywords:

analysis, stakeholder, evidence, problem, objective, requirement, functional requirement, actor, use case, system boundary, include, extend.

Final message:

Good use case diagrams begin with good requirements.